AgenticTrip

Multi-Agent Autonomous Travel Planning System

---

1. Project Vision

AgenticTrip is a multi-agent autonomous travel planning and execution system powered by AI Agents, conversational memory, orchestration workflows, Retrieval-Augmented Generation (RAG), and future MCP-based tool execution.

The system is designed to:

- Understand user travel goals
- Break large travel tasks into smaller subtasks
- Allow multiple AI agents to collaborate
- Plan complete trips autonomously
- Execute travel-related workflows step-by-step
- Integrate external APIs and MCP tools in future phases

---

2. Long-Term Goal

Build an enterprise-grade autonomous AI travel assistant capable of:

- Planning trips
- Comparing transportation options
- Booking flights/buses/trains
- Reserving hotels
- Booking cabs
- Planning itineraries
- Managing budgets
- Coordinating travel agents
- Maintaining conversational memory
- Performing autonomous execution workflows

---

3. Initial Use Cases

Example 1 — Goa Trip

User:

"Plan my Goa trip for 5 days from Pune."

Expected Flow:

1. Understand destination
2. Ask budget/preferences
3. Find travel options
4. Compare flights/buses/trains
5. Suggest hotels
6. Plan itinerary
7. Arrange cabs
8. Finalize booking plan
9. Return complete travel plan

Example 2 — Shimla Trip

User:

"Plan a Shimla family trip for 4 people."

Expected Flow:

- Multi-agent collaboration
- Travel optimization
- Budget planning
- Family-friendly hotel recommendations
- Local guide suggestions
- Full itinerary generation

---

4. Core Architecture

```
User
 ↓
Supervisor Agent
 ↓
--------------------------------
| Travel Agent                |
| Hotel Agent                 |
| Booking Agent               |
| Cab Agent                   |
| Tourism Agent               |
| Expense Agent               |
--------------------------------
 ↓
Tools / APIs / MCP
 ↓
Execution
```

5. Current Technical Stack

Layer	Technology
Frontend	React + Vite
Backend	FastAPI
LLM	Azure OpenAI GPT-4.1
Embeddings	text-embedding-3-small
Vector Store	FAISS-like local vector DB
Memory	Custom Conversation Memory
Architecture	Modular Agentic Architecture
UI	Enterprise Chat UI
Streaming	FastAPI Streaming Responses

6. Existing Completed Work

Completed Foundation

Backend

- FastAPI backend setup
- Azure OpenAI integration
- Embedding generation
- RAG pipeline
- Multi-document retrieval
- Vector search
- Source citations
- Streaming responses

Frontend

- React + Vite frontend
- Enterprise chat UI
- Streaming response rendering
- Source citation UI

Agent Architecture

- Tool abstraction
- HR Agent
- Agent orchestrator
- Conversational memory

- Stateful conversation handling
- Memory-aware prompting

---

7. Existing Folder Structure

```
backend/app/
│
├── agents/
│   └── hr_agent.py
│
├── tools/
│   └── rag_search_tool.py
│
├── memory/
│   └── conversation_memory.py
│
├── orchestrator/
│   └── agent_orchestrator.py
```

---

8. Next Evolution — Multi-Agent System

The next milestone is evolving from:

```
Single AI Agent
```

To:

```
Multi-Agent Collaborative System
```

---

9. Planned Agent Structure

```
agents/
│
├── supervisor_agent.py
├── travel_agent.py
├── hotel_agent.py
├── booking_agent.py
├── cab_agent.py
```

```
├── tourism_agent.py
├── expense_agent.py
```

---

10. Planned Agent Responsibilities

Supervisor Agent

Responsibilities:

- Understand user goals
- Decompose tasks
- Delegate work to other agents
- Merge final responses
- Manage workflow state

Travel Agent

Responsibilities:

- Compare travel options
- Flights
- Buses
- Trains
- Road trip planning

Hotel Agent

Responsibilities:

- Hotel recommendations
- Budget filtering
- Location analysis
- Ratings and amenities

Booking Agent

Responsibilities:

- Coordinate bookings
- Simulate confirmations
- Manage booking state
- Future MCP integrations

Cab Agent

Responsibilities:

- Airport pickup/drop
- Local transport planning
- Cab coordination

Tourism Agent

Responsibilities:

- Tourist attractions
- Local guides
- Food recommendations
- Itinerary planning

Expense Agent

Responsibilities:

- Budget planning
- Cost optimization
- Expense breakdowns

---

11. Planned Tool Architecture

```
tools/
│
├── flight_tool.py
├── hotel_tool.py
├── bus_tool.py
├── train_tool.py
├── weather_tool.py
├── tourism_tool.py
├── maps_tool.py
├── booking_tool.py
```

---

12. Planned Project Phases

PHASE 1 — Foundation

Status: COMPLETED

Features:

- RAG
- Memory
- Streaming
- Agent architecture
- Azure OpenAI integration
- Tool abstraction
- Orchestration

PHASE 2 — Multi-Agent Collaboration

Status: Upcoming

Features:

- Supervisor Agent
- Agent communication
- Task delegation
- Shared memory
- Workflow orchestration

PHASE 3 — Tool Routing

Status: Upcoming

Features:

- Dynamic tool selection
- Tool execution pipelines
- Agent-to-tool interaction
- Workflow branching

PHASE 4 — Mock Execution System

Status: Upcoming

Features:

- Simulated bookings
- Mock flight reservations
- Mock hotel bookings
- Fake confirmation IDs
- Local JSON persistence

Purpose:

- Learn orchestration before real APIs

PHASE 5 — Long-Term Memory

Status: Upcoming

Features:

- Persistent sessions
- User preferences
- Travel history
- Stateful planning

PHASE 6 — MCP Integration

Status: Future

Features:

- MCP servers
- Tool interoperability
- Standardized tool calling
- Cross-agent communication

PHASE 7 — Real API Integrations

Status: Future

Possible APIs:

Feature	API
Flights	Amadeus

Feature	API
Hotels	Booking.com
Maps	OpenStreetMap
Weather	OpenWeather
Tourism	Google Places

---

13. MCP (Model Context Protocol)

MCP will eventually enable:

- Standardized tool access
- Dynamic tool discovery
- Agent interoperability
- Cross-system communication
- External workflow execution

Official MCP Reference: https://modelcontextprotocol.io/introduction

---

14. Key Learning Objectives

This project is intended to build expertise in:

- AI Agent Architecture
- Multi-Agent Systems
- Conversational Memory
- Workflow Orchestration
- Tool Routing
- Retrieval-Augmented Generation
- Autonomous AI Workflows
- MCP-based integrations
- Enterprise AI Engineering

---

15. Engineering Philosophy

AgenticTrip will be developed incrementally.

Priority Order:

1. Architecture
2. Intelligence
3. Orchestration

4. Memory
5. Tooling
6. Mock execution
7. Real integrations

The project will focus on learning core AI engineering principles before introducing expensive APIs or production-scale infrastructure.

---

16. Current Project Status

Current State:

```
Conversational RAG-based Enterprise AI Agent
```

Next Major Milestone:

```
Supervisor-based Multi-Agent Collaboration System
```

---

17. Future Vision

Long-term AgenticTrip aims to evolve into:

- Autonomous travel planner
- AI execution engine
- Multi-agent orchestration platform
- MCP-compatible workflow system
- Enterprise-grade AI operations platform

---

18. Project Authoring Journey

The project started as:

```
Enterprise AI Support Copilot
```

Then evolved into:

```
Conversational AI Agent
```

And is now evolving toward:

```
Autonomous Multi-Agent Agentic Workflow System
```

END OF DOCUMENT